

Relatório de Auditoria de Segurança

Paróquia São Sebastião de Altônia

24 de setembro de 2026

7 achados — 1 crítica · 2 alta · 1 média · 2 baixa · 1 informativa

Escopo auditado

Repositório **paroquiaaltonia**, na íntegra: os 20 route handlers de `app/api/**` e `app/payload-api/**`, as 4 Server Actions de `app/actions.ts`, as 11 collections e 6 globals do Payload (incluindo o access control de cada uma), o frontend em `app/(frontend)/**` e `components/**`, os arquivos de configuração (`next.config.mjs`, `payload.config.ts`, `vercel.json`, `.env.example`), o histórico completo do git e o bundle compilado do cliente. Verificações de confirmação foram feitas contra o ambiente de produção usando apenas requisições de leitura.

Stack detectada

Linguagem	TypeScript 5 / Node.js
Framework	Next.js 16.3.3 (App Router, React 19, Server Components e Server Actions)
CMS / ORM	Payload CMS 3.88 sobre @payloadcms/db-postgres (Drizzle); acesso SQL direto via `pg` para o schema legado `bot`
Banco	PostgreSQL (Neon, plano gratuito) — dois bancos: o do Payload e o legado
Autenticação	Payload Auth (JWT em cookie) na collection `users`, painel montado em /cms. Não há papéis: o modelo é binário — autenticado no CMS ou anônimo
Frontend	React 19 + Tailwind CSS v4, PWA com service worker e Web Push (VAPID)
Deploy / CI	Vercel (região gru1), vercel.json; GitHub Actions em .github/. Sem Docker, Helm ou Terraform
Serviços externos	Groq (LLM), Resend (e-mail), Vercel Blob (mídia), YouTube Data API, liturgia.up.railway.app

Nota metodológica

As cinco categorias pedidas foram traduzidas para os equivalentes desta stack antes da varredura. Onde uma categoria não tem correspondente no projeto, isso está dito explicitamente em vez de forçar um achado.

1. Banco sem tranca (isolamento)

O projeto não é multi-inquilino e não usa Supabase/RLS. O isolamento real acontece em duas camadas: o **access control por collection do Payload** (collections/*.ts) e a **redação campo a campo** feita manualmente nas rotas públicas (ex.: app/api/velas/publicas/route.ts). Para conteúdo anônimo (velas), o "dono" é provado por um **ownershipToken** (UUID) em vez de um usuário. A auditoria mapeou as duas camadas e procurou coleções cujo access control não acompanha a promessa de privacidade da interface.

2. Permissão definida no navegador

Não existe nenhum gate de papel no frontend — a varredura por isAdmin/canEdit/role/permission não achou nenhum. O equivalente nesta stack é: **operações privilegiadas expostas como rota de API ou Server Action**. Cada uma foi cruzada com a verificação de autorização correspondente no servidor.

3. IDOR

Percorridos **todos os 20 route handlers** de app/api/** e app/payload-api/**, além das 4 Server Actions de app/actions.ts. Foco nas rotas que recebem identificador por path, query ou body.

4. Chaves expostas

Varredura do código-fonte, configs, next.config.mjs, vercel.json, .env.example, scripts e do **histórico completo do git (272 commits)**. Também foi inspecionado o **bundle compilado do cliente** (.next/static) atrás de valores reais de chave. Sem Docker/Helm/CI com segredos para auditar.

5. Inputs sem tratamento (XSS)

Levantados todos os usos de dangerouslySetInnerHTML, innerHTML, eval e new Function; verificado se a lib de sanitização já presente no projeto (lib/sanitize.ts, DOMPurify) é aplicada em cada um. No backend, verificada a interpolação de entrada do usuário em HTML de e-mail e em prompts.

Categoria sem correspondente no projeto

Permissão definida no navegador não se aplica na forma clássica. O projeto não tem papéis: a collection users (collections/Users.ts) não declara nenhum campo de papel, e a varredura do frontend por isAdmin, canEdit, role ou permission não encontrou nenhum gate de interface para cruzar com o backend. Só existem dois níveis — autenticado no /cms ou anônimo. A categoria foi então auditada na forma que existe aqui: operações privilegiadas expostas como rota HTTP ou Server Action sem verificação equivalente no servidor.

Sem infraestrutura como código para auditar. Não há Dockerfile, docker-compose, chart Helm nem Terraform no repositório; o deploy é gerenciado pela Vercel e o único arquivo é vercel.json, que contém apenas a região de execução.

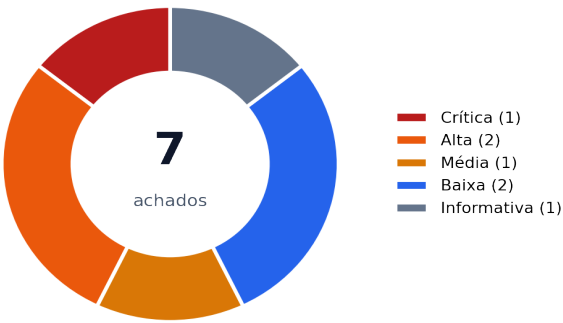
Resumo executivo

A auditoria registrou **7 achados**, sendo **3 de severidade crítica ou alta**, e **10 controles verificados e aprovados**. O achado central é uma exposição de dados pessoais já ocorrendo em produção: fotos e nomes de fiéis que pediram sigilo ao acender uma vela estão publicamente listáveis. Os demais riscos altos são um ponto de XSS alimentado por API de terceiro e uma rota de IA aberta que gasta a chave da paróquia.

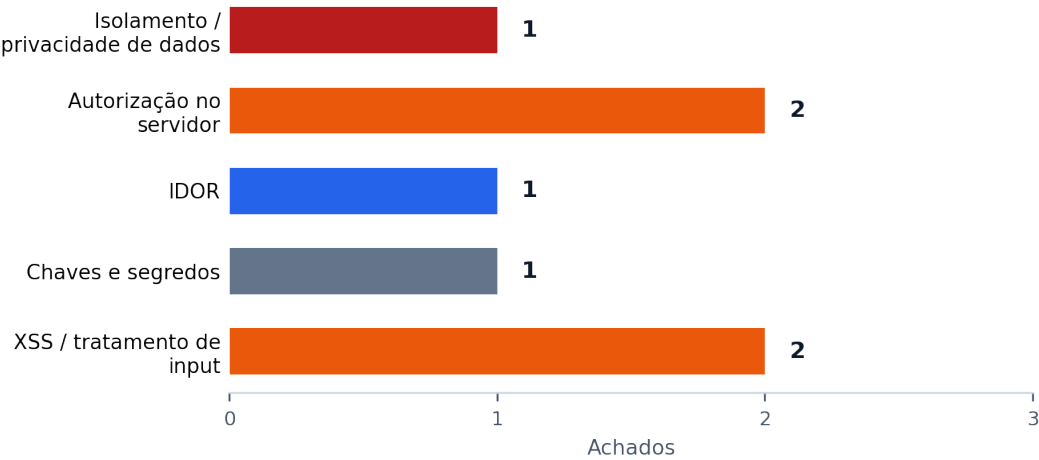
O contraste é importante para interpretar o relatório: o projeto acerta os controles difíceis — posse verificada em todo handler por ID, SQL inteiramente parametrizado, coleções com dado pessoal fechadas, nenhum segredo no código ou no histórico. As falhas estão nas bordas, onde uma defesa existente deixou de ser estendida a um caminho novo.

Estado desta revisão: 6 dos 7 achados já foram corrigidos e verificados; o restante está registrado como decisão consciente. A linha **Situação**, ao fim de cada achado, diz o que foi feito e com qual commit. As correções ainda precisam ser publicadas para valerem em produção.

Severidade	Achados	O que representa
CRÍTICA	1	Dado pessoal exposto agora, sem necessidade de autenticação
ALTA	2	Execução de script de terceiro no domínio e abuso de chave paga
MÉDIA	1	Operação privilegiada sem autorização própria, hoje protegida por acaso
BAIXA	2	Endurecimento: exige condição prévia para ser explorado
INFORMATIVA	1	Padrão frágil sem impacto explorável hoje



Distribuição dos achados por severidade



Achados por categoria (cor = maior severidade da categoria)

Pontos fortes verificados

Cada item abaixo foi conferido no código e, quando indicado, confirmado contra produção. Esta seção também serve de prova de cobertura da auditoria.

OK	Posse verificada em todos os handlers de vela As três rotas que operam uma vela por ID exigem o ownershipToken e comparam com o documento antes de agir: <code>app/api/velas/[id]/apagar/route.ts:20</code> , <code>app/api/velas/[id]/editar/route.ts:27</code> e <code>app/api/velas/[id]/minha/route.ts:22</code> . Nenhuma confia no ID sozinho. É o padrão correto de IDOR para conteúdo anônimo.
OK	Collections com dado pessoal fecham a leitura anônima <code>velas</code> (<code>collections/Velas.ts:17-20</code>), <code>intencoes</code> (<code>collections/Intencoes.ts:12-17</code>) e <code>avisos</code> (<code>collections/Avisos.ts:25-28</code>) exigem <code>req.user</code> nas quatro operações, e <code>velas</code> ainda bloqueia <code>create</code> . Confirmado em produção: <code>/payload-api/velas</code> e <code>/payload-api/intencoes</code> devolvem 403.
OK	Redação de campos privados documento a documento <code>app/api/velas/publicas/route.ts:39-47</code> monta a resposta manualmente e devolve null para nome, intenção e foto conforme as flags de cada vela, em vez de repassar o documento. Verificado em produção: a vela ativa sai com nome e foto nulos.
OK	SQL integralmente parametrizado Todas as consultas ao schema legado usam placeholders (<code>\$1</code> , <code>\$2...</code>). As poucas interpolações em template string são constantes numéricas definidas no próprio código (<code>RATE_LIMIT_WINDOW_MIN</code> , <code>LIMITE_PADRAO</code> , <code>JANELA_DIAS</code>), nunca entrada do usuário. Nenhuma injeção de SQL encontrada.
OK	Defesa em profundidade nos formulários públicos As duas rotas públicas de escrita combinam limite de 3 envios por IP a cada 10 minutos contabilizado antes da validação, campo honeypot e tempo mínimo de preenchimento: <code>app/api/velas/route.ts:7-13,36-50</code> e <code>app/api/intencoes/route.ts:6-13,57-70</code> . Upload público valida tipo e tamanho no servidor (<code>app/api/velas/_shared.ts:28-37</code>).
OK	Rotas de cron autenticadas por segredo <code>app/api/cron/check-live-mass/route.ts:13-22</code> e <code>app/api/cron/check-velas-expiradas/route.ts:15-24</code> exigem <code>CRON_SECRET</code> em produção e recusam subir sem ele (500), em vez de liberar por omissão.
OK	Rota administrativa valida a sessão no servidor <code>app/api/admin/transmissao-ao-vivo/route.ts:17-21</code> chama <code>payload.auth({ headers: req.headers })</code> e devolve 401 sem usuário — a verificação não depende de o botão estar escondido no painel.
OK	Nenhum segredo no código, no histórico ou no bundle <code>.gitignore</code> ignora <code>.env*</code> e permite só <code>.env.example</code> ; a varredura dos 272 commits do histórico não achou nenhum arquivo de ambiente já commitado; <code>.env.example</code> traz apenas placeholders (<code>postgresql://user:password@host/db</code>). No bundle compilado (<code>.next/static</code>) não há nenhum valor real — nem <code>vercel_blob_rw_</code> , nem <code>gsk_</code> , nem <code>re_</code> , nem <code>npg_</code> . A única ocorrência de <code>BLOB_READ_WRITE_TOKEN</code> é o nome da variável dentro do SDK da Vercel.
OK	Sanitização aplicada na saída da IA <code>components/explicacao-leitura.tsx:93</code> passa o HTML gerado a partir da resposta do modelo por <code>sanitizeRichText()</code> (<code>lib/sanitize.ts</code> , <code>DOMPurify</code> com <code>allowlist</code> de tags e atributos) antes de injetar no DOM.
OK	Resposta personalizada marcada como não cacheável <code>app/api/notificacoes/route.ts:26-29</code> força <code>Cache-Control: no-store, private</code> , impedindo que um intermediário sirva o histórico de um aparelho para outro — o padrão do Next ali seria público.

Pontos fracos — os riscos centrais

- O modelo de privacidade das velas é aplicado em duas camadas (access control e redação por campo), mas a foto escapa das duas por viver em outra collection, de leitura pública — e o nome viaja junto no texto

alternativo.

- As defesas contra abuso (rate limit, honeypot) existem e funcionam, mas não foram estendidas à rota mais cara do sistema, que é a que consome chave de terceiro.
- A sanitização de HTML existe no projeto e é aplicada em um ponto, mas não nos onze pontos que recebem conteúdo de fora do domínio.
- Server Actions são tratadas como funções internas, sem verificação de autorização própria — a segurança hoje depende de ninguém importá-las de um componente cliente.

Achados detalhados

Organizados por categoria. Cada achado traz arquivo e linha exatos, o trecho de código, por que é explorável, o impacto e a correção sugerida. Condições de explorabilidade, quando existem, estão marcadas em **Condição**.

C1 — Isolamento / privacidade de dados

Severidade	Arquivo:linha	Descrição
CRÍTICA	collections/Media.ts:5-7 app/api/velas/route.ts:86 app/api/velas/[id]/editar/route.ts:54	F-01 — Collection `media` com leitura pública expõe fotos e nomes de velas marcados como privados

F-01 — Collection `media` com leitura pública expõe fotos e nomes de velas marcados como privados

CRÍTICA

```
collections/Media.ts:5-7
```

```
    access: {
      read: () => true,
    },
```

```
app/api/velas/route.ts:84-88
```

```
    const media = await payload.create({
      collection: "media",
      data: { alt: `Foto da vela de ${nome}` },
      file: await fileParaPayloadFile(foto),
    })
```

Por que é explorável: A funcionalidade de velas promete sigilo: quem acende escolhe esconder o nome (nomePrivado), a intenção (intencaoPrivada) e a foto (fotoPrivada), e a interface mostra "Só você vê esta informação". Essa promessa é cumprida em duas camadas — a collection `velas` recusa leitura anônima e /api/velas/publicas redige campo a campo. Mas a foto não é guardada em `velas`: é um documento da collection `media`, que tem leitura pública irrestrita. Pior, o texto alternativo é montado como `Foto da vela de \${nome}`, então o nome que a pessoa pediu para esconder fica gravado no próprio documento público.

Condição / evidência: Confirmado em produção em 24/09/2026. GET /payload-api/velas e /payload-api/intencoes devolvem 403 ("You are not allowed to perform this action"), e /api/velas/publicas devolve a única vela ativa com nome e foto redigidos (nome: null, foto: null). Já GET /payload-api/media?limit=50 devolve 200 com 18 documentos, entre eles seis fotos de vela cujo alt segue o padrão "Foto da vela de <nome>", cada uma com a URL do arquivo e um nome de pessoa real legível. Os nomes foram omitidos deste relatório de propósito — publicá-los repetiria o próprio vazamento. Os documentos afetados são os de id 10, 11, 12, 15, 16 e 17. Nenhuma autenticação é necessária. ALCANCE MEDIDO depois, ao corrigir (a primeira versão deste relatório superestimava): das 14 velas já acesas, 4 têm foto e 9 marcaram nomePrivado. Das 4 com foto, DUAS marcaram nomePrivado — ou seja, duas pessoas pediram explicitamente para esconder o nome e ele foi publicado assim mesmo. Quanto à foto: uma única vela marcou fotoPrivada, e ela não tem imagem anexada. Portanto nenhuma foto chegou a ser exposta contra a escolha de quem a enviou — essa metade da falha era estrutural e ainda não tinha sido acionada. A metade do nome, sim, foi consumada.

Impacto: Quebra direta da promessa de privacidade, com dado pessoal real de paroquianos já exposto: o nome de quem acendeu a vela, incluindo o de duas pessoas que marcaram explicitamente para escondê-lo. Qualquer pessoa na internet enumerava a collection inteira com uma requisição. A exposição das fotos era igualmente possível, mas ainda não tinha acontecido. Como se trata de dado pessoal sensível por contexto (devoção religiosa, muitas vezes ligada a doença ou luto), há também exposição da paróquia perante a LGPD.

Correção sugerida: 1) Parar de escrever o nome no alt: usar um texto neutro como "Foto enviada com uma vela". 2) Restringir a leitura de `media` — ou separando as fotos de vela em uma collection própria com `read: ({ req }) => Boolean(req.user)` e servindo a foto pública apenas pela rota que já redige (/api/velas/publicas), ou aplicando um access control por documento em `media` que esconda os arquivos vinculados a velas privadas. 3) Corrigir o alt dos seis

documentos já gravados e avaliar a remoção das fotos de velas encerradas.

Situação: CORRIGIDO (commit 6e911a7). O alt passou a ser neutro, `media` ganhou o campo `privado` alimentado por fotoPrivada, e o access.read devolve um Where em vez de booleano. Os seis documentos já gravados tiveram o alt corrigido no banco. Verificado com um documento marcado como privado: some da listagem e a URL direta do arquivo devolve 403, enquanto os arquivos públicos seguem em 200.

C2 — Autorização no servidor

Severidade	Arquivo:linha	Descrição
ALTA	app/api/liturgia/explicacao/route.ts: 21-22 app/api/liturgia/explicacao/route.ts: 34-52 app/api/liturgia/explicacao/route.ts: 25, 54-57	F-03 — Proxy de LLM aberto: /api/liturgia/explicacao sem autenticação, sem rate limit e com prompt controlado pelo cliente
MÉDIA	app/actions.ts:1, 103-110 collections/Avisos.ts:57-58	F-04 — Server Action privilegiada sendNotificationToAll não verifica autorização

F-03 — Proxy de LLM aberto: /api/liturgia/explicacao sem autenticação, sem rate limit e com prompt controlado pelo cliente

ALTA

```
app/api/liturgia/explicacao/route.ts:21-22

export async function POST(request: NextRequest) {
  const { tipo, referencia, titulo, texto } = await request.json()

  app/api/liturgia/explicacao/route.ts:34-39

  const prompt = `Como um padre católico experiente, forneça uma explicação pastoral e didática da seguinte ${tipo}:

  Referência: ${referencia}
  Título: ${titulo}

  Texto: ${texto}

  app/api/liturgia/explicacao/route.ts:54-57

  const response = await fetch("https://api.groq.com/openai/v1/chat/completions", {
    method: "POST",
    headers: {
      Authorization: `Bearer ${apiKey}`,
```

Por que é explorável: A rota aceita POST de qualquer origem, sem sessão, sem token e sem limite de requisições, e repassa quatro campos controlados pelo cliente direto para dentro do prompt enviado à Groq com a chave da paróquia. Não há validação de tipo, tamanho ou conteúdo: `texto` pode ser um prompt inteiro que sobrescreve a instrução de sistema. Na prática, a rota é um proxy de LLM gratuito e anônimo custeado pela conta da paróquia.

Condição / evidência: O contraste interno prova que a defesa é conhecida no projeto e só não foi aplicada aqui: as outras duas rotas públicas de escrita têm limite de 3 envios por IP a cada 10 minutos gravado em bot.velas_rate_limit / bot.intencoes_rate_limit, campo honeypot e trava de tempo mínimo de preenchimento (app/api/velas/route.ts:7-13,36-50 e app/api/intencoes/route.ts:6-10,57-70).

Impacto: Esgotamento da cota da Groq (derrubando a funcionalidade para os fiéis), custo financeiro se o plano for pago, e uso da identidade da paróquia para gerar conteúdo arbitrário — inclusive ofensivo — por meio de injeção de prompt. Também serve de amplificador de negação de serviço, já que cada requisição mantém uma função serverless ocupada por segundos.

Correção sugerida: Aplicar o mesmo rate limit por IP já usado nas outras rotas públicas; validar tipo e tamanho máximo de `tipo`, `referencia`, `titulo` e `texto`; e, de preferência, aceitar apenas uma referência litúrgica (ex.: "Mt 5,1-12") e montar o

texto no servidor a partir da liturgia do dia, em vez de aceitar texto livre do cliente.

Situação: CORRIGIDO (commit 0f2d1a0). Limite de seis pedidos por IP a cada trinta minutos, contabilizado antes da validação, mais validação de tipo e tamanho dos quatro campos e delimitação do texto no prompt. A tabela `bot.explicacao_rate_limit` foi criada. De passagem, corrigido o 500 do bloco `catch`.

F-04 — Server Action privilegiada `sendNotificationToAll` não verifica autorização

MÉDIA

```
app/actions.ts:1
```

```
"use server"
```

```
app/actions.ts:103-108
```

```
export async function sendNotificationToAll(
  title: string,
  body: string,
  url = "/",
  opcoes: OpcoesNotificacao = {}
) {
```

Por que é explorável: `app/actions.ts` é um módulo "use server": cada função exportada vira um endpoint POST que o Next registra e expõe por um identificador de ação. `sendNotificationToAll` dispara notificação push com título, corpo e URL arbitrários para TODAS as inscrições da tabela `bot.push_subscriptions`, e o arquivo inteiro não contém nenhuma verificação de autenticação — confirmado por varredura (`payload.auth`, `req.user`, `headers()`). A proteção hoje é acidental: a ação só é chamada de código de servidor (hooks das collections, `live-mass-bot`, `cron`).

Condição / evidência: Condição de explorabilidade verificada: o identificador da ação NÃO está no bundle do cliente (`sendNotificationToAll` não aparece em `.next/static`), porque nenhum componente cliente a importa. Já `subscribe` e `unsubscribe` são importadas por `hooks/use-push-subscription.ts:4`, então os identificadores DELAS estão publicados. Ou seja, o risco vira exploração direta no dia em que alguém importar essa ação de um componente cliente, ou se o manifesto de ações vaziar.

Impacto: Quem conseguir o identificador da ação envia notificação push em nome da paróquia para todos os aparelhos inscritos, com URL de destino arbitrária — vetor de phishing de alta credibilidade (ex.: "Atualize os dados do seu dízimo" apontando para um site falso).

Correção sugerida: Verificar a sessão no início de `sendNotificationToAll` com `payload.auth({ headers: await headers() })` e recusar sem usuário. Para as chamadas internas que não têm sessão (`cron` e `hooks`), extrair a lógica para uma função comum não exportada e deixar exportada apenas a versão que checa autorização.

Situação: CORRIGIDO (commit f72a2e5). A lógica de envio foi para `app/lib/push-broadcast.ts`, que não é "use server"; a action exportada confere a sessão do CMS antes de delegar, e os cinco pontos internos passaram a usar a função comum.

C3 — IDOR

Severidade	Arquivo:linha	Descrição
BAIXA	<code>app/api/notificacoes/route.ts:13-14, 21</code> <code>app/lib/notification-log.ts:111-137</code>	F-05 — Histórico de notificações é servido com base em identificador enviado pelo próprio cliente

F-05 — Histórico de notificações é servido com base em identificador enviado pelo próprio cliente

BAIXA

```
app/api/notificacoes/route.ts:13-14
```

```
const deviceId = req.nextUrl.searchParams.get("deviceId")
const endpoint = req.nextUrl.searchParams.get("endpoint")
```

```
app/lib/notification-log.ts:129-133
    AND (
      (device_id IS NULL AND endpoint IS NULL)
      OR ($2::text IS NOT NULL AND device_id = $2)
      OR ($3::text IS NOT NULL AND endpoint = $3)
    )
```

Por que é explorável: O deviceId chega por query string e é usado direto no WHERE, sem prova de posse. Quem souber o deviceId de outra pessoa lê as notificações individuais dela — por exemplo o aviso "Sua vela apagou — A vela que você acendeu por <nome>", que revela um nome possivelmente marcado como privado.

Condição / evidência: Mitigadores verificados: o deviceId é gerado com crypto.randomUUID() e guardado só no localStorage (lib/device-id.ts:17), então não é adivinhável nem enumerável; a rota define Cache-Control: no-store, private (app/api/notificacoes/route.ts:28), evitando que um intermediário sirva a resposta de um aparelho para outro; e o próprio código documenta que isso não é autenticação. A exploração exige vazamento prévio do UUID (XSS, acesso físico ou log).

Impacto: Leitura do histórico de avisos individuais de um aparelho específico, incluindo nomes ligados a velas privadas. Depende de conhecer o UUID.

Correção sugerida: Tratar o deviceId como segredo de portador e combiná-lo com o ownershipToken quando o aviso for ligado a uma vela, ou assinar o deviceId no servidor (HMAC) na primeira gravação e exigir a assinatura na leitura. No mínimo, remover o nome do corpo da notificação quando nomePrivado estiver marcado — hoje isso já é feito em app/api/cron/check-velas-expiradas/route.ts:45, então basta manter a coerência.

Situação: PARCIAL. A metade que expunha nome já estava correta e foi verificada: doc.nome só aparece dentro do ternário de nomePrivado, e título, url e opções não carregam nada do documento. O deviceId segue sem prova de posse — decisão mantida, dado que é um UUID aleatório e a resposta não é cacheável.

C4 — Chaves e segredos

Severidade	Arquivo:linha	Descrição
INFORMATIVA	payload.config.ts:70	F-07 — PAYLOAD_SECRET com fallback para string vazia, sem validação explícita de inicialização

F-07 — PAYLOAD_SECRET com fallback para string vazia, sem validação explícita de inicialização

INFORMATIVA

```
payload.config.ts:70
    secret: process.env.PAYLOAD_SECRET || "",
```

Por que é explorável: O padrão `|| ""` é exatamente a forma que costuma virar segredo real quando não sobrescrito, já que PAYLOAD_SECRET assina o JWT de sessão do painel. Neste caso específico, porém, a verificação mostrou que NÃO há vulnerabilidade: o Payload recusa subir com segredo falsy.

Condição / evidência: Verificado em payload/dist/index.js:319-320 — `if (!this.config.secret) { throw new Error('Error: missing secret key. A secret key is needed to secure Payload.')}`. Como a string vazia é falsy, a aplicação falha alto em vez de assinar sessões com chave previsível. O fallback é, na prática, um no-op que apenas troca undefined por "".

Impacto: Nenhum impacto explorável hoje. Fica registrado porque o padrão é frágil: basta uma atualização do Payload afrouxar essa checagem para o fallback virar uma chave de assinatura previsível.

Correção sugerida: Remover o `|| ""` e validar na inicialização, falhando com mensagem clara: `if (!process.env.PAYLOAD\_SECRET) throw new Error("PAYLOAD\_SECRET não configurada")`. Vale estender a mesma validação a CRON_SECRET, VAPID_PRIVATE_KEY e DATABASE_URL.

Situação: CORRIGIDO (commit 5451ddd). Os fallbacks para string vazia foram removidos e a validação roda no primeiro import real, sendo pulada durante o build para não quebrar a análise estática.

C5 — XSS / tratamento de input

Severidade	Arquivo:linha	Descrição
ALTA	app/(frontend)/liturgia/liturgia-content.tsx:220, 230, 250, 280, 298, 322, 346, 369, 377, 385, 395 app/(frontend)/liturgia/liturgia-content.tsx:67 app/(frontend)/liturgia/page.tsx:27	F-02 — Onze pontos de dangerouslySetInnerHTML recebem HTML de API de terceiro sem sanitização
BAIXA	components/json-ld.tsx:6 app/(frontend)/page.tsx:84-108 app/(frontend)/eventos/[slug]/page.tsx:95-101	F-06 — JSON-LD injetado sem escapar a sequência de fechamento de script

F-02 — Onze pontos de dangerouslySetInnerHTML recebem HTML de API de terceiro sem sanitização

ALTA

app/(frontend)/liturgia/liturgia-content.tsx:250

```
<div
  dangerouslySetInnerHTML={{ __html: formatarTextoComSobrescrito(leitura.texto) }}
/>
```

app/(frontend)/liturgia/liturgia-content.tsx:105-107

```
const formatarTextoComSobrescrito = (texto: string) => {
  // Converte TODOS os números para sobrescrito
  const textoFormatado = texto.replace(/\d+/g, (numero) => {
```

app/(frontend)/liturgia/page.tsx:27

```
const res = await fetch("https://liturgia.up.railway.app/v2/", { next: { revalidate: 3600 } })
```

Por que é explorável: Todo o conteúdo litúrgico do dia (antífonas, leituras, salmo, evangelho e orações) vem de liturgia.up.railway.app, um serviço de terceiro sobre o qual a paróquia não tem controle, e é injetado cru no DOM. A única transformação aplicada, formatarTextoComSobrescrito, apenas troca dígitos por caracteres sobrescritos — não escapa nem remove marcação. Se a API for comprometida, trocar de dono ou simplesmente passar a devolver HTML, qualquer <script> ou atributo de evento no texto executa no domínio da paróquia.

Condição / evidência: O projeto JÁ TEM a defesa pronta e não a usa aqui: lib/sanitize.ts expõe sanitizeRichText() com DOMPurify e allowlist de tags, e ela é corretamente aplicada na explicação gerada por IA (components/explicacao-leitura.tsx:93). Os onze pontos da liturgia ficaram de fora.

Impacto: XSS armazenado de fato, refletido para todos os visitantes da página de liturgia — uma das mais acessadas do site. Permite roubo de sessão de quem estiver logado no /cms, desfiguração da página e redirecionamento de fiéis para páginas falsas de dízimo ou PIX.

Correção sugerida: Aplicar sanitizeRichText() nos onze pontos, exatamente como já é feito em explicacao-leitura.tsx. Como o texto litúrgico não precisa de marcação rica, o ideal é uma allowlist ainda mais estreita (apenas , <i>,
, <p>). Alternativa mais forte: sanitizar no servidor, em liturgia/page.tsx, para que nem o HTML inicial carregue conteúdo não confiável.

Situação: CORRIGIDO (commit 8ab5427). O escape passou a ser feito na entrada de formatarTextoComSobrescrito, o que cobre os onze pontos sem alterar as chamadas.

F-06 — JSON-LD injetado sem escapar a sequência de fechamento de script

BAIXA

components/json-ld.tsx:6

```
return <script type="application/ld+json" dangerouslySetInnerHTML={{ __html:
JSON.stringify(data) }} />
```

Por que é explorável: JSON.stringify não escapa "<", então um valor que contenha a sequência de fechamento de tag script encerra o bloco e o que vier depois é interpretado como HTML. Os dados vêm do CMS: título e descrição de evento, telefone, e-mail e endereço do global contact-info.

Condição / evidência: Explorabilidade limitada: todos os campos que alimentam o JSON-LD só podem ser escritos por usuário autenticado no /cms (as collections e globals correspondentes exigem req.user para escrita). Não há caminho anônimo. É, portanto, endurecimento contra editor malicioso ou conta comprometida, não uma porta aberta.

Impacto: Um editor com acesso ao CMS — ou alguém que comprometa uma conta — consegue executar script em todas as páginas que carregam aquele dado estruturado, incluindo a home.

Correção sugerida: Escapar antes de injetar: JSON.stringify(data).replace(/</g, "\\u003c"). É uma linha em components/json-ld.tsx e cobre todos os usos, já que o componente é o ponto único.

Situação: CORRIGIDO (commit 7908041).

Recomendações priorizadas

Prioridade	Ação	Detalhe	Achados
P1	Fechar a exposição das fotos e nomes de velas	Trocar o alt por texto neutro, restringir a leitura da collection media (ou separar as fotos de vela) e corrigir os seis documentos já publicados. É o único achado com dados pessoais reais já expostos.	F-01
P1	Sanitizar o conteúdo litúrgico antes de injetar no DOM	Aplicar sanitizeRichText() nos onze pontos de app/(frontend)/liturgia/liturgia-content.tsx, preferencialmente com allowlist estreita, ou sanitizar no servidor em liturgia/page.tsx.	F-02
P2	Proteger a rota de explicação da IA	Rate limit por IP igual ao das demais rotas públicas, validação de tamanho e tipo dos campos e, idealmente, montar o texto no servidor a partir da referência.	F-03
P2	Exigir sessão na Server Action de broadcast	Verificar payload.auth no início de sendNotificationToAll e mover a lógica sem verificação para uma função interna usada por cron e hooks.	F-04
P3	Endurecer os pontos de menor risco	Escapar "<" no JSON-LD, amarrar o histórico de notificações a uma prova de posse e remover o fallback de PAYLOAD_SECRET trocando-o por validação de inicialização.	F-05, F-06, F-07

Issues para o GitHub

Texto completo de cada issue, em Markdown, pronto para copiar e colar. Os três achados de endurecimento (F-05, F-06 e F-07) foram agrupados numa issue só, para não gerar ruído no repositório.

```

--- ISSUE 1 ---

**Título:** [Segurança] Collection `media` com leitura pública expõe fotos e nomes de velas
marcados como privados

**Labels:** `security`, `severity:critica`

## Problema

A funcionalidade de velas promete sigilo: quem acende escolhe esconder o nome (nomePrivado), a
intenção (intencaoPrivada) e a foto (fotoPrivada), e a interface mostra "Só você vê esta
informação". Essa promessa é cumprida em duas camadas – a collection `velas` recusa leitura
anônima e /api/velas/publicas redige campo a campo. Mas a foto não é guardada em `velas`: é um
documento da collection `media`, que tem leitura pública irrestrita. Pior, o texto alternativo
é montado como `Foto da vela de ${nome}`, então o nome que a pessoa pediu para esconder fica
gravado no próprio documento público.

## Evidência

`collections/Media.ts:5-7`

```ts
access: {
 read: () => true,
},
```

`app/api/velas/route.ts:84-88`

```ts
const media = await payload.create({
 collection: "media",
 data: { alt: `Foto da vela de ${nome}` },
 file: await fileParaPayloadFile(foto),
})
```

- `collections/Media.ts:5-7`
- `app/api/velas/route.ts:86`
- `app/api/velas/[id]/editar/route.ts:54`

## Condição de explorabilidade

Confirmado em produção em 24/09/2026. GET /payload-api/velas e /payload-api/intencoes devolvem
403 ("You are not allowed to perform this action"), e /api/velas/publicas devolve a única vela
ativa com nome e foto redigidos (nome: null, foto: null). Já GET /payload-api/media?limit=50
devolve 200 com 18 documentos, entre eles seis fotos de vela cujo alt segue o padrão "Foto da
vela de <nome>", cada uma com a URL do arquivo e um nome de pessoa real legível. Os nomes
foram omitidos deste relatório de propósito – publicá-los repetiria o próprio vazamento. Os
documentos afetados são os de id 10, 11, 12, 15, 16 e 17. Nenhuma autenticação é necessária.

ALCANCE MEDIDO depois, ao corrigir (a primeira versão deste relatório superestimava): das 14
velas já acesas, 4 têm foto e 9 marcaram nomePrivado. Das 4 com foto, DUAS marcaram
nomePrivado – ou seja, duas pessoas pediram explicitamente para esconder o nome e ele foi
publicado assim mesmo. Quanto à foto: uma única vela marcou fotoPrivada, e ela não tem imagem
anexada. Portanto nenhuma foto chegou a ser exposta contra a escolha de quem a enviou – essa
metade da falha era estrutural e ainda não tinha sido acionada. A metade do nome, sim, foi
consumada.

## Impacto

Quebra direta da promessa de privacidade, com dado pessoal real de paroquianos já exposto: o

```

ISSUE 1 (continuação 2/2)

nome de quem acendeu a vela, incluindo o de duas pessoas que marcaram explicitamente para escondê-lo. Qualquer pessoa na internet enumerava a collection inteira com uma requisição. A exposição das fotos era igualmente possível, mas ainda não tinha acontecido. Como se trata de dado pessoal sensível por contexto (devoção religiosa, muitas vezes ligada a doença ou luto), há também exposição da paróquia perante a LGPD.

Correção sugerida

1) Parar de escrever o nome no alt: usar um texto neutro como "Foto enviada com uma vela". 2) Restringir a leitura de `media` – ou separando as fotos de vela em uma collection própria com `read: ({ req }) => Boolean(req.user)` e servindo a foto pública apenas pela rota que já redige (/api/velas/publicas), ou aplicando um access control por documento em `media` que esconda os arquivos vinculados a velas privadas. 3) Corrigir o alt dos seis documentos já gravados e avaliar a remoção das fotos de velas encerradas.

Situação

CORRIGIDO (commit 6e911a7). O alt passou a ser neutro, `media` ganhou o campo `privado` alimentado por fotoPrivada, e o access.read devolve um Where em vez de booleano. Os seis documentos já gravados tiveram o alt corrigido no banco. Verificado com um documento marcado como privado: some da listagem e a URL direta do arquivo devolve 403, enquanto os arquivos públicos seguem em 200.

Critérios de aceite

- [] GET /payload-api/media sem autenticação não devolve nenhum documento cujo alt comece com "Foto da vela"
- [] Nenhum documento de media guarda o nome de quem acendeu a vela no campo alt
- [] Os seis documentos já publicados (ids 10, 11, 12, 15, 16, 17) tiveram o alt corrigido
- [] Uma vela com fotoPrivada marcada não tem a imagem acessível por nenhuma rota anônima
- [] Uma vela sem fotoPrivada continua aparecendo normalmente em /api/velas/publicas

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

****Título:** [Segurança] Onze pontos de dangerouslySetInnerHTML recebem HTML de API de terceiro sem sanitização**

****Labels:** `security`, `severity:alta`**

Problema

Todo o conteúdo litúrgico do dia (antífonas, leituras, salmo, evangelho e orações) vem de `liturgia.up.railway.app`, um serviço de terceiro sobre o qual a paróquia não tem controle, e é injetado cru no DOM. A única transformação aplicada, `formatarTextoComSobrescrito`, apenas troca dígitos por caracteres sobrescritos – não escapa nem remove marcação. Se a API for comprometida, trocar de dono ou simplesmente passar a devolver HTML, qualquer `<script>` ou atributo de evento no texto executa no domínio da paróquia.

Evidência

`app/(frontend)/liturgia/liturgia-content.tsx:250`

```
```ts
<div
 dangerouslySetInnerHTML={{ __html: formatarTextoComSobrescrito(leitura.texto) }}
/>
```
```

`app/(frontend)/liturgia/liturgia-content.tsx:105-107`

```
```ts
const formatarTextoComSobrescrito = (texto: string) => {
 // Converte TODOS os números para sobrescrito
 const textoFormatado = texto.replace(/\d+/g, (numero) => {
 ...
 })
}
```

`app/(frontend)/liturgia/page.tsx:27`

```
```ts
const res = await fetch("https://liturgia.up.railway.app/v2/", { next: { revalidate: 3600 } })
```
```

- `app/(frontend)/liturgia/liturgia-content.tsx:220, 230, 250, 280, 298, 322, 346, 369, 377, 385, 395`
- `app/(frontend)/liturgia/liturgia-content.tsx:67`
- `app/(frontend)/liturgia/page.tsx:27`

### ## Condição de explorabilidade

O projeto JÁ TEM a defesa pronta e não a usa aqui: `lib/sanitize.ts` expõe `sanitizeRichText()` com `DOMPurify` e `allowlist` de tags, e ela é corretamente aplicada na explicação gerada por IA (`components/explicacao-leitura.tsx:93`). Os onze pontos da liturgia ficaram de fora.

### ## Impacto

XSS armazenado de fato, refletido para todos os visitantes da página de liturgia – uma das mais acessadas do site. Permite roubo de sessão de quem estiver logado no `/cms`, desfiguração da página e redirecionamento de fiéis para páginas falsas de dízimo ou PIX.

### ## Correção sugerida

Aplicar `sanitizeRichText()` nos onze pontos, exatamente como já é feito em `explicacao-leitura.tsx`. Como o texto litúrgico não precisa de marcação rica, o ideal é uma `allowlist` ainda mais estreita (apenas `<b>`, `<i>`, `<br>`, `<p>`). Alternativa mais forte: sanitizar

ISSUE 2 (continuação 2/2)

no servidor, em `liturgia/page.tsx`, para que nem o HTML inicial carregue conteúdo não confiável.

## Situação

CORRIGIDO (commit 8ab5427). O escape passou a ser feito na entrada de `formatarTextoComSobrescrito`, o que cobre os onze pontos sem alterar as chamadas.

## Critérios de aceite

- [ ] Os onze pontos de `dangerouslySetInnerHTML` em `liturgia-content.tsx` passam por `sanitizeRichText()` ou equivalente
- [ ] Uma resposta forjada da API de liturgia contendo `<img src=x onerror=alert(1)>` não executa script na página
- [ ] O texto litúrgico continua renderizando com os números em sobrescrito, como hoje
- [ ] Existe teste cobrindo o caso de HTML malicioso vindo da API externa

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

**\*\*Título:\*\*** [Segurança] Proxy de LLM aberto: /api/liturgia/explicacao sem autenticação, sem rate limit e com prompt controlado pelo cliente

**\*\*Labels:\*\*** `security`, `severity:alta`

### ## Problema

A rota aceita POST de qualquer origem, sem sessão, sem token e sem limite de requisições, e repassa quatro campos controlados pelo cliente direto para dentro do prompt enviado à Groq com a chave da paróquia. Não há validação de tipo, tamanho ou conteúdo: `texto` pode ser um prompt inteiro que sobrescreve a instrução de sistema. Na prática, a rota é um proxy de LLM gratuito e anônimo custeado pela conta da paróquia.

### ## Evidência

```
`app/api/liturgia/explicacao/route.ts:21-22`
```

```
````ts
export async function POST(request: NextRequest) {
  const { tipo, referencia, titulo, texto } = await request.json()
  ...
```

```
`app/api/liturgia/explicacao/route.ts:34-39`
```

```
````ts
const prompt = `Como um padre católico experiente, forneça uma explicação pastoral e didática da seguinte ${tipo}:`
```

```
Referência: ${referencia}
Título: ${titulo}
```

```
Texto: ${texto}
...`
```

```
`app/api/liturgia/explicacao/route.ts:54-57`
```

```
````ts
const response = await fetch("https://api.groq.com/openai/v1/chat/completions", {
  method: "POST",
  headers: {
    Authorization: `Bearer ${apiKey}`,
    ...
```

```
- `app/api/liturgia/explicacao/route.ts:21-22`
- `app/api/liturgia/explicacao/route.ts:34-52`
- `app/api/liturgia/explicacao/route.ts:25, 54-57`
```

Condição de explorabilidade

O contraste interno prova que a defesa é conhecida no projeto e só não foi aplicada aqui: as outras duas rotas públicas de escrita têm limite de 3 envios por IP a cada 10 minutos gravado em `bot.velas_rate_limit` / `bot.intencoes_rate_limit`, campo honeypot e trava de tempo mínimo de preenchimento (`app/api/velas/route.ts:7-13,36-50` e `app/api/intencoes/route.ts:6-10,57-70`).

Impacto

Esgotamento da cota da Groq (derrubando a funcionalidade para os fiéis), custo financeiro se o plano for pago, e uso da identidade da paróquia para gerar conteúdo arbitrário – inclusive ofensivo – por meio de injeção de prompt. Também serve de amplificador de negação de serviço, já que cada requisição mantém uma função serverless ocupada por segundos.

ISSUE 3 (continuação 2/2)

Correção sugerida

Aplicar o mesmo rate limit por IP já usado nas outras rotas públicas; validar tipo e tamanho máximo de `tipo`, `referencia`, `titulo` e `texto`; e, de preferência, aceitar apenas uma referência litúrgica (ex.: "Mt 5,1-12") e montar o texto no servidor a partir da liturgia do dia, em vez de aceitar texto livre do cliente.

Situação

CORRIGIDO (commit 0f2d1a0). Limite de seis pedidos por IP a cada trinta minutos, contabilizado antes da validação, mais validação de tipo e tamanho dos quatro campos e delimitação do texto no prompt. A tabela bot.explicacao_rate_limit foi criada. De passagem, corrigido o 500 do bloco catch.

Critérios de aceite

- [] POST /api/liturgia/explicacao acima do limite por IP devolve 429
- [] Campos tipo, referencia, titulo e texto têm tamanho máximo validado e rejeitam payload fora do limite com 400
- [] Requisição com tipo não string não derruba a rota com 500 no bloco catch
- [] A cota da Groq não é consumida por requisição anônima em volume

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

****Título:**** [Segurança] Server Action privilegiada sendNotificationToAll não verifica autorização

****Labels:**** `security`, `severity:media`

Problema

app/actions.ts é um módulo "use server": cada função exportada vira um endpoint POST que o Next registra e expõe por um identificador de ação. sendNotificationToAll dispara notificação push com título, corpo e URL arbitrários para TODAS as inscrições da tabela bot.push_subscriptions, e o arquivo inteiro não contém nenhuma verificação de autenticação – confirmado por varredura (payload.auth, req.user, headers()). A proteção hoje é acidental: a ação só é chamada de código de servidor (hooks das collections, live-mass-bot, cron).

Evidência

`app/actions.ts:1`

```
``ts
"use server"
``
```

`app/actions.ts:103-108`

```
``ts
export async function sendNotificationToAll(
  title: string,
  body: string,
  url = "/",
  opcoes: OpcoesNotificacao = {}
) {
  ``
```

- `app/actions.ts:1, 103-110`
- `collections/Avisos.ts:57-58`

Condição de explorabilidade

Condição de explorabilidade verificada: o identificador da ação NÃO está no bundle do cliente (sendNotificationToAll não aparece em .next/static), porque nenhum componente cliente a importa. Já subscribe e unsubscribe são importadas por hooks/use-push-subscription.ts:4, então os identificadores DELAS estão publicados. Ou seja, o risco vira exploração direta no dia em que alguém importar essa ação de um componente cliente, ou se o manifesto de ações vazar.

Impacto

Quem conseguir o identificador da ação envia notificação push em nome da paróquia para todos os aparelhos inscritos, com URL de destino arbitrária – vetor de phishing de alta credibilidade (ex.: "Atualize os dados do seu dízimo" apontando para um site falso).

Correção sugerida

Verificar a sessão no início de sendNotificationToAll com payload.auth({ headers: await headers() }) e recusar sem usuário. Para as chamadas internas que não têm sessão (cron e hooks), extrair a lógica para uma função comum não exportada e deixar exportada apenas a versão que checa autorização.

Situação

CORRIGIDO (commit f72a2e5). A lógica de envio foi para app/lib/push-broadcast.ts, que não é "use

ISSUE 4 (continuação 2/2)

server"; a action exportada confere a sessão do CMS antes de delegar, e os cinco pontos internos passaram a usar a função comum.

Critérios de aceite

- [] sendNotificationToAll recusa execução sem sessão válida do CMS
- [] O cron e os hooks das collections continuam enviando notificação normalmente
- [] Nenhuma função exportada de app/actions.ts executa operação privilegiada sem verificar autorização

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

****Título:**** [Segurança] Endurecimento: escape do JSON-LD, posse do histórico de notificações e validação de segredos

****Labels:**** `security`, `severity:baixa`

Problema

Três itens de endurecimento de baixo risco, agrupados por serem correções pequenas e independentes entre si – não justificam uma issue cada.

F-05 – Histórico de notificações é servido com base em identificador enviado pelo próprio cliente

O deviceId chega por query string e é usado direto no WHERE, sem prova de posse. Quem souber o deviceId de outra pessoa lê as notificações individuais dela – por exemplo o aviso "Sua vela apagou – A vela que você acendeu por <nome>", que revela um nome possivelmente marcado como privado.

****Evidência:****

`app/api/notificacoes/route.ts:13-14`

```
```ts
const deviceId = req.nextUrl.searchParams.get("deviceId")
const endpoint = req.nextUrl.searchParams.get("endpoint")
```
```

`app/lib/notification-log.ts:129-133`

```
```ts
AND (
 (device_id IS NULL AND endpoint IS NULL)
 OR ($2::text IS NOT NULL AND device_id = $2)
 OR ($3::text IS NOT NULL AND endpoint = $3)
)
```
```

****Condição:**** Mitigadores verificados: o deviceId é gerado com crypto.randomUUID() e guardado só no localStorage (lib/device-id.ts:17), então não é adivinhável nem enumerável; a rota define Cache-Control: no-store, private (app/api/notificacoes/route.ts:28), evitando que um intermediário sirva a resposta de um aparelho para outro; e o próprio código documenta que isso não é autenticação. A exploração exige vazamento prévio do UUID (XSS, acesso físico ou log).

****Impacto:**** Leitura do histórico de avisos individuais de um aparelho específico, incluindo nomes ligados a velas privadas. Depende de conhecer o UUID.

****Correção:**** Tratar o deviceId como segredo de portador e combiná-lo com o ownershipToken quando o aviso for ligado a uma vela, ou assinar o deviceId no servidor (HMAC) na primeira gravação e exigir a assinatura na leitura. No mínimo, remover o nome do corpo da notificação quando nomePrivado estiver marcado – hoje isso já é feito em app/api/cron/check-velas-expiradas/route.ts:45, então basta manter a coerência.

F-06 – JSON-LD injetado sem escapar a sequência de fechamento de script

JSON.stringify não escapa "<", então um valor que contenha a sequência de fechamento de tag script encerra o bloco e o que vier depois é interpretado como HTML. Os dados vêm do CMS: título e descrição de evento, telefone, e-mail e endereço do global contact-info.

****Evidência:****

ISSUE 5 (continuação 2/2)

```
`components/json-ld.tsx:6`
```

```
``ts
return <script type="application/ld+json" dangerouslySetInnerHTML={{ __html:
  JSON.stringify(data) }} />
``
```

****Condição:**** Explorabilidade limitada: todos os campos que alimentam o JSON-LD só podem ser escritos por usuário autenticado no /cms (as collections e globals correspondentes exigem req.user para escrita). Não há caminho anônimo. É, portanto, endurecimento contra editor malicioso ou conta comprometida, não uma porta aberta.

****Impacto:**** Um editor com acesso ao CMS – ou alguém que comprometa uma conta – consegue executar script em todas as páginas que carregam aquele dado estruturado, incluindo a home.

****Correção:**** Escapar antes de injetar: `JSON.stringify(data).replace(/</g, "\\u003c")`. É uma linha em `components/json-ld.tsx` e cobre todos os usos, já que o componente é o ponto único.

F-07 – PAYLOAD_SECRET com fallback para string vazia, sem validação explícita de inicialização

O padrão ``|| ""`` é exatamente a forma que costuma virar segredo real quando não sobrescrito, já que `PAYLOAD_SECRET` assina o JWT de sessão do painel. Neste caso específico, porém, a verificação mostrou que NÃO há vulnerabilidade: o Payload recusa subir com segredo falsy.

****Evidência:****

```
`payload.config.ts:70`
```

```
``ts
secret: process.env.PAYLOAD_SECRET || "",
``
```

****Condição:**** Verificado em `payload/dist/index.js:319-320` – ``if (!this.config.secret) { throw new Error('Error: missing secret key. A secret key is needed to secure Payload.') }``. Como a string vazia é falsy, a aplicação falha alto em vez de assinar sessões com chave previsível. O fallback é, na prática, um no-op que apenas troca undefined por "".

****Impacto:**** Nenhum impacto explorável hoje. Fica registrado porque o padrão é frágil: basta uma atualização do Payload afrouxar essa checagem para o fallback virar uma chave de assinatura previsível.

****Correção:**** Remover o ``|| ""`` e validar na inicialização, falhando com mensagem clara: ``if (!process.env.PAYLOAD_SECRET) throw new Error("PAYLOAD_SECRET não configurada")``. Vale estender a mesma validação a `CRON_SECRET`, `VAPID_PRIVATE_KEY` e `DATABASE_URL`.

Critérios de aceite

- [] (F-05) Ler o histórico de outro aparelho exige mais do que conhecer o deviceId
- [] (F-05) A notificação de vela apagada não inclui o nome quando nomePrivado está marcado
- [] (F-06) Um valor de CMS contendo a sequência de fechamento de script não quebra o bloco JSON-LD
- [] (F-06) O dado estruturado continua válido no teste de resultados ricos do Google
- [] (F-07) A aplicação recusa iniciar com mensagem clara quando `PAYLOAD_SECRET` não está definida
- [] (F-07) Não há mais fallback silencioso para string vazia em segredo

--- FIM ISSUE 5 ---